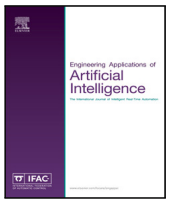
Engineering Applications of Artificial Intelligence 182 (2026) 116074

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Research paper

Fortifying smart contracts through rich heterogeneous graph semantics and dual attention guards☆

Yingying Qu [a,b], Jiangtao Cui [a], Haotian Chi [c], Yonggang Li [a], Yeh-Ching Chung [d], Shunrong Jiang [a,b,e,*]

[a] School of computer Science and Technology/School of Artificial Intelligence, China University of Mining and Technology, Xuzhou, 221116, China
[b] Mine Digitization Engineering Research Center of the Ministry of Education, Xuzhou, 221116, China
[c] School of Automation and Software Engineering, Shanxi University, Taiyuan, 030006, China
[d] School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, Shenzhen, 518172, China
[e] Jiangsu Provincial Industrial Technology Engineering Center for Intelligent Sensing and Emergency IoT in Underground Space, Xuzhou, 221116, China

ARTICLE INFO

Keywords:
Smart contracts
Vulnerability detection
Heterogeneous graphs
Dual attention mechanism

ABSTRACT

Smart contract vulnerabilities pose significant security risks in blockchain applications, often leading to severe financial losses. Existing graph-based detection methods, particularly those based on heterogeneous graph neural networks, still suffer from limited semantic modeling and insufficient exploitation of graph heterogeneity. To address these challenges, we propose a novel framework based on rich heterogeneous graph representations and a dual-attention mechanism. Specifically, our method constructs a comprehensive heterogeneous graph by integrating control flow, data dependencies, function calls, and contract inheritance, enabling more expressive semantic modeling. Furthermore, a dual-attention mechanism is designed to capture both intra-type and inter-type node importance, allowing fine-grained feature learning across heterogeneous structures. Experimental results on a benchmark dataset demonstrate that our method consistently outperforms twelve state-of-the-art methods across seven common vulnerability types. In particular, our method achieves the best F1-scores on both contract-level and line-level detection tasks, with performance gains of up to 34.02% compared to baseline methods. Moreover, it consistently attains F1-scores around or above 95% across most of the seven vulnerability types. The computational complexity of our method scales approximately linearly with the graph size, making it applicable to large-scale smart contract analysis. These results validate the effectiveness and general applicability of the proposed method in smart contract vulnerability detection.

1. Introduction

Smart contracts (Buterin et al., 2013) are self-executing programs deployed on the blockchain, enabling users to enforce predefined contractual logic without relying on a trusted third party. This capability facilitates various applications, including automated transactions, decentralized finance (DeFi), and supply chain management. However, once a smart contract is deployed, its content becomes immutable. This means that if vulnerabilities exist within the contract, they cannot be fixed in a timely manner, potentially leading to significant financial losses (Bresil et al., 2025). The impact of these vulnerabilities far exceeds that of traditional software bugs, due to the irreversible nature of blockchain transactions and their financial implications. As a result, it is crucial to perform vulnerability detection before the deployment of smart contracts to mitigate these risks (Liang et al., 2025).

Existing smart contract vulnerability detection methods can be categorized into two main approaches: traditional methods (Feist et al., 2019; Jiang et al., 2018; Grieco et al., 2020; Tikhomirov et al., 2018) and neural network-based methods (Tann et al., 2018; Nguyen et al., 2022, 2023; Chen et al., 2023; Liu et al., 2023; Xu et al., 2024). Traditional methods primarily rely on static analysis, dynamic analysis, and formal verification techniques (Jiang et al., 2018). These approaches depend on predefined rules or patterns, requiring extensive domain expertise and manual intervention (Tann et al., 2018). Moreover, as smart contracts grow in size and complexity, dynamic analysis and formal verification are less practical for them (Zhang and

☆ A preliminary version was presented at IEEE GLOBECOM, 2025:2916-2921 (Qu et al., 2025).

* Corresponding author at: School of computer Science and Technology/School of Artificial Intelligence, China University of Mining and Technology, Xuzhou, 221116, China.
E-mail addresses: ts23170078p31@cumt.edu.cn (Y. Qu), 08222230@cumt.edu.cn (J. Cui), htchi@sxu.edu.cn (H. Chi), liyg@cumt.edu.cn (Y. Li), ychung@cuhk.edu.cn (Y.-C. Chung), jsywow@gmail.com (S. Jiang).

https://doi.org/10.1016/j.engappai.2026.116074
Received 15 January 2026; Received in revised form 26 May 2026; Accepted 20 August 2026

Liu, 2022). To address these limitations, neural network-based methods are proposed, which can be broadly classified into sequence-based models and graph-based models. Sequence-based models convert smart contract bytecode or source code into sequential data and apply models such as LSTMs (Gers et al., 2000) or Transformers (Vaswani et al., 2017) to capture semantic and contextual information. However, they overlook the structural relationships within the code and struggle with long sequences, often losing critical information (Chen et al., 2023). In contrast, graph-based models represent smart contracts as various graph structures, such as control flow graphs (CFGs), data flow graphs (DFGs), and program dependence graphs (PDGs) (Sun et al., 2023). These representations are then processed using graph neural networks (GNNs) to learn node-level features (Wang et al., 2025), effectively preserving the structural information of the contract.

Unfortunately, existing GNN-based methods still have several limitations (Ren et al., 2025). Homogeneous graph models consider only a single type of node and edge, making it difficult to capture the diverse entities and complex interactions within smart contracts, and thus struggle to identify intricate vulnerability patterns effectively (Li et al., 2026). Some heterogeneous graph-based approaches attempt to address this by introducing multiple node and edge types; however, they often focus on a subset of relationships, such as control flow or function calls, while overlooking other critical dependencies like data flow and contract inheritance. In addition, most existing methods treat all nodes as equally important during feature extraction, ignoring both inter-type differences and intra-type variations, which limits the expressiveness and generalization of the learned representations. In summary, current methods leave three key challenges largely unaddressed: they provide limited representation of heterogeneous entities and relationships, fail to fully capture important semantic dependencies, and lack mechanisms to distinguish the contributions of different node types. Addressing these gaps is essential for improving the accuracy and comprehensiveness of smart contract vulnerability detection.

To address these drawbacks, we propose RHDA, a method based on rich heterogeneous graph representations and a dual attention mechanism for smart contract vulnerability detection. RHDA aims to construct a more comprehensive heterogeneous graph representation of smart contracts and fully exploit the structural information of different node types during feature extraction. By doing so, it enhances the model's ability to identify vulnerability patterns effectively.

Overall, RHDA makes the following key contributions:

- We propose RHDA, a novel heterogeneous graph modeling framework for smart contract vulnerability detection, which jointly captures multiple semantic relationships, including control flow, data dependencies, function calls, and contract inheritance. Unlike existing approaches that rely on homogeneous graph representations or limited structural information, RHDA provides a more comprehensive and fine-grained modeling of contract semantics.
- We design a dual-attention mechanism that operates at both intra-type and inter-type levels, enabling the model to adaptively learn the importance of nodes within the same type as well as across different types. This hierarchical attention design effectively enhances the model's ability to identify critical vulnerability patterns in complex heterogeneous structures.
- Extensive experiments on seven representative vulnerability types demonstrate that RHDA consistently outperforms twelve state-of-the-art methods. In particular, it achieves the best F1-scores on both contract-level and line-level detection tasks, with performance gains of up to 34.02% compared to baseline methods. Moreover, RHDA consistently attains F1-scores around or above 95% across most vulnerability types, and maintains stable performance across different graph scales, demonstrating its robustness and generalization capability.

The structure of this thesis is organized as follows. Section 2 presents the preliminary knowledge. In Section 3, we provide a detailed description of the proposed RHDA method and its core techniques. Section 4 evaluates the effectiveness and superiority of the proposed approach through extensive experiments. Section 5 reviews related work in the field. Finally, Section 7 concludes the thesis.

2. Preliminary

2.1. Smart contract

Smart contracts are self-executing protocols built on blockchain technology. They gained widespread attention and practical application with the emergence of Ethereum, which provides a decentralized platform specifically designed to facilitate and execute smart contracts. The core feature of smart contracts lies in their ability to autonomously enforce agreement terms through decentralization, thereby eliminating the need for a trusted third party (Szabo, 1996). Modern smart contracts are primarily written in domain-specific languages such as Solidity and Vyper. These contracts are compiled into bytecode that runs on the Ethereum Virtual Machine (EVM) (Buterin, 2014) and are deployed on distributed ledger platforms like Ethereum. They play a vital role in enabling DeFi applications and decentralized applications (DApps).

2.2. Smart contract vulnerabilities

Although the immutability of blockchain ensures the reliability of contract execution, it also implies that once a contract is deployed, its code cannot be modified. This significantly amplifies the risk posed by potential security vulnerabilities (Atzei et al., 2017). Here, we focus on seven common and severe vulnerabilities frequently found in real-world smart contracts. Access Control (AC) vulnerabilities often result from missing or improper permission checks, leading to unauthorized access or data leakage (Nguyen et al., 2023). Arithmetic (AR) bugs, such as integer overflows and underflows, may cause abnormal calculations and financial losses, and thus require strict boundary checks (Xu et al., 2024). Denial-of-Service (DoS) attacks exhaust resources (e.g., gas or computation), rendering critical functionalities unavailable and degrading system availability (Samreen and Alalfi, 2021). Front Running (FR) exploits the transparency of transactions by inserting malicious ones before others, gaining unfair profit, especially in decentralized exchanges (Crincoli et al., 2022). Reentrancy (RE) vulnerabilities manipulate control flow through recursive external calls, which can be exploited to drain contract funds (Nguyen et al., 2023). Timestamp Manipulation (TM) leverages unreliable time sources like block timestamps to alter contract behavior (Fei et al., 2023). Unchecked Low-Level Calls (ULLC) stem from failing to verify return values of low-level operations such as `send()`, potentially causing fund loss or inconsistent contract states (Crincoli et al., 2022). These vulnerabilities highlight the critical need for comprehensive, lifecycle-oriented security protection in smart contract development.

2.3. Heterogeneous graph learning

Heterogeneous Graphs (HGs) are graph structures designed to model multiple types of nodes and multiple types of edges simultaneously. Formally, a heterogeneous graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \tau, \phi)$, where $\mathcal{V}$ is the set of nodes, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. The mapping function $\tau : \mathcal{V} \to \mathcal{A}$ assigns each node to a specific node type, while $\phi : \mathcal{E} \to \mathcal{R}$ maps each edge to its corresponding edge type. Here, $\mathcal{A}$ and $\mathcal{R}$ denote the sets of all node types and edge types, respectively. Compared to traditional homogeneous graphs, heterogeneous graphs can more accurately represent complex systems involving multiple entities and diverse relationships (Xu et al., 2024). Due to this advantage, they

have been widely used in various domains such as knowledge graphs, recommendation systems, and program analysis.

Heterogeneous Graph Neural Networks (HGNNs) are a class of deep learning models designed to handle graph structures containing multiple types of nodes and edges, enabling them to capture the semantic differences among various entities and relations more effectively than traditional homogeneous GNNs (Zhao et al., 2024). For example, the Heterogeneous Graph Transformer (HGT) (Hu et al., 2020) is based on the Transformer architecture. It introduces a meta-path-aware attention mechanism to handle the complex interactions among different types of nodes and edges in heterogeneous graphs. Specifically, HGT designs independent projection and attention parameters for each node and edge type. This allows the model to distinguish structural and semantic roles associated with various types. As a result, it can better capture the rich information inherent in heterogeneous graphs. Furthermore, the meta-path awareness enables HGT to preserve contextual relevance across heterogeneous relations. Therefore, it provides a more expressive and flexible framework for learning on heterogeneous graph data.

2.4. Attention mechanism

Attention Mechanism is a neural network framework that mimics the human cognitive ability to focus selectively on relevant information. Its core idea lies in dynamically assigning weights to different inputs, thereby emphasizing their relative importance (Niu et al., 2021). Unlike traditional models that process all inputs uniformly, attention allows the model to concentrate on the most relevant parts. This enhances both feature representation and contextual modeling. The computation typically involves the interaction among three components: query, key, and value. Attention weights are computed by measuring the similarity between the query and each key. These weights are then used to aggregate the corresponding values in a weighted manner. This mechanism has been widely adopted in the Transformer (Han et al., 2021) architecture, where variants such as multi-head attention further improve its expressiveness.

3. Methodology

3.1. Overview

In this paper, we propose RHDA (as depicted in Fig. 1), which is composed of three modules: (1) smart contract heterogeneous graph construction (SCHGC) module, (2) heterogeneous graph feature extraction (HGFE) module, and (3) vulnerability detection (VD) module. Specifically, SCHGC converts the smart contract source code into a rich heterogeneous graph containing various types of nodes and edges, and performs an initial embedding of the node information. HGFE then employs a multi-layer heterogeneous graph neural network to extract node-level features from the embedded graph. It further utilizes a dual attention mechanism to capture the contribution of both same-type nodes and different-type nodes in the context of vulnerability detection, thereby deriving graph-level features. Finally, VD module leverages the extracted node and graph features to determine the presence of vulnerabilities at both the line and contract levels. The subsequent sections provide a detailed exposition of each module.

3.2. SCHGC module

SCHGC module consists of two steps. First, it transforms the smart contract into a heterogeneous graph with rich features, providing a more comprehensive feature representation for vulnerability detection. Second, it performs initial node embedding on the smart contract heterogeneous graph, which helps incorporate smart contract syntax information and enables subsequent feature extraction by the heterogeneous graph neural network.

Algorithm 1: RECONSTRUCT CFG

Input: Smart contract file $\mathcal{S}$ with SLITHER
Output: Node set $\mathcal{V}$, edge set $\mathcal{E}$

1 Initialize node set $\mathcal{V} \leftarrow \emptyset$, edge set $\mathcal{E} \leftarrow \emptyset$;
2 Create a node v_S to represent contract structure S;
3 $\mathcal{V} \leftarrow \mathcal{V} \cup \{v_S\}$;
4 **foreach** *contract* $c \in \mathcal{S}$ **do**
5 Create a node v_c to represent contract c;
6 $\mathcal{V} \leftarrow \mathcal{V} \cup \{v_c\}$;
7 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_c, v_S, \texttt{AGGREGATE})\}$;
8 Create nodes for state variables of c and aggregate them;
9 **foreach** *function* $f \in c$ **do**
10 Create a node v_f to represent function f;
11 $\mathcal{V} \leftarrow \mathcal{V} \cup \{v_f\}$;
12 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_f, v_c, \texttt{AGGREGATE})\}$;
13 Create nodes for parameters of f and aggregate them;
14 IntraFuncDFS($\mathcal{V}, \mathcal{E}, f.$firstNode$, v_f,$ NEXT$, v_f$);
15 **end**
16 **end**
17 **foreach** *function* f *and called function* cf **do**
18 Locate nodes v_f and v_{cf} in $\mathcal{V}$;
19 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_{cf}, v_f, \texttt{BE\_CALLED})\}$;
20 **end**
21 **foreach** *contract* c *and inherited contract* ic **do**
22 Locate nodes v_c and v_{ic} in $\mathcal{V}$;
23 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_{ic}, v_c, \texttt{BE\_INHERITED})\}$;
24 **end**

3.2.1. Feature-rich heterogeneous graph construction

Building upon the control flow graph (CFG), we construct a heterogeneous graph by integrating three key relationships: data dependency, function calls, and contract inheritance. These relationships capture variable interactions, inter-functional information flows, and hierarchical contract structures, respectively, aiding in the detection of various vulnerabilities (Zhang and Liu, 2022). To achieve this, SCHGC processes smart contract source files using the Slither (Feist et al., 2019) tool, extracting the basic contract structure and control flow. It then enhances this representation by incorporating the three relationships while retaining only vulnerability-relevant information, such as code statements, function names, visibility, and variable properties.

Algorithm 1 describes the specific process of reconstructing the CFG based on the contract structure parsed by Slither. The input is the smart contract file $\mathcal{S}$ after Slither analysis, and the output is the node set $\mathcal{V}$ and the edge set $\mathcal{E}$ of the heterogeneous graph. First, the algorithm initializes $\mathcal{V}$ and $\mathcal{E}$ and creates a node v_S to represent the smart contract file S (lines **1–3**). Then, it iterates through each contract c in $\mathcal{S}$, generating a corresponding contract node v_c and linking it to v_S via an `AGGREGATE` edge, which semantically indicates that the contract is a component of the overall contract file structure (lines **4–7**). Next, it processes the state variables of c (line **8**), followed by each function f (lines **9–12**) and its parameters (line **13**) similarly. For each function f, the algorithm performs a depth-first search using the `IntraFuncDFS` algorithm (**Algorithm 2**) to explore the internal control flow, reconstruct execution sequences, and incorporate data dependencies (line **14**). Afterward, it identifies the functions cf called by each function f and establishes `CALL` edges accordingly, where each `CALL` edge captures an function invocation that reflects the dynamic execution relationship between caller and callee (lines **17–20**). Finally, it detects inherited contracts ic for each contract c and adds corresponding `INHERITANCE` edges (lines **21–24**).

The implementation of `IntraFuncDFS`, invoked in **Algorithm 1**, is detailed in **Algorithm 2**. The input includes the node set $\mathcal{V}$ and edge set $\mathcal{E}$ from **Algorithm 1**, the current Slither control flow node n, its parent heterogeneous graph node v_p, the edge type e_t between n and v_p, and the `FUNCTION` node v_f to which n belongs. First, the algorithm

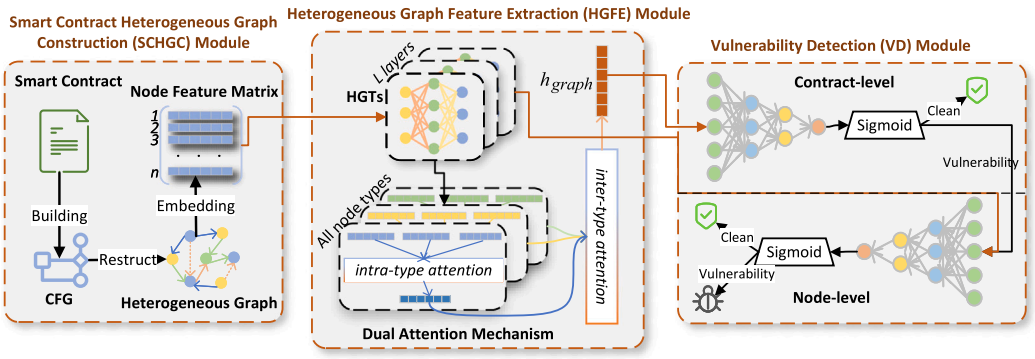


Fig. 1. RHDA.

Algorithm 2: IntraFuncDFS

Input: Node set $\mathcal{V}$, edge set $\mathcal{E}$;
Slither function child node n;
Parent node v_p and parent edge type e_t;
Function node v_f to which n belongs
Output: Updated node set $\mathcal{V}$, edge set $\mathcal{E}$

1 Create a node v_n to represent node n with $type(n)$;
2 $\mathcal{V} \leftarrow \mathcal{V} \cup \{v_n\}$;
3 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_p, v_n, e_t)\}$;
4 $\mathcal{E} \leftarrow \mathcal{E} \cup \{(v_n, v_f, \texttt{AGGREGATE})\}$;
5 **foreach** *variable var* ***read*** *by n* **do**
6 $\quad \mathcal{E} \leftarrow \mathcal{E} \cup \{(var, v_n, \texttt{BE\_READED})\}$;
7 **end**
8 **foreach** *variable var* ***written*** *by n* **do**
9 $\quad \mathcal{E} \leftarrow \mathcal{E} \cup \{(v_n, var, \texttt{WRITE})\}$;
10 **end**
11 **if** *n has child node(s)* **then**
12 $\quad$ **if** $type(n) \in \{IF, IFLOOP\}$ **then**
13 $\quad\quad$ IntraFuncDFS($\mathcal{V}, \mathcal{E}, n.\texttt{trueChild}, v_n$, IF_TRUE, v_f);
14 $\quad\quad$ IntraFuncDFS($\mathcal{V}, \mathcal{E}, n.\texttt{falseChild}, v_n$, IF_FALSE, v_f);
15 $\quad$ **end**
16 $\quad$ **else**
17 $\quad\quad$ IntraFuncDFS($\mathcal{V}, \mathcal{E}, n.\texttt{child}, v_n$, NEXT, v_f);
18 $\quad$ **end**
19 **end**

creates a heterogeneous graph node v_n to represent n based on its type $type(n)$ (line **1**). Table 1 provides descriptions of different node types. Then, it establishes an edge between v_n and its parent node v_p and further aggregates v_n into the FUNCTION node v_f (lines **2–4**). Next, the algorithm constructs data dependency relationships. It identifies the variables *var* that n reads from and writes to, adding READ and WRITE edges between *var* and v_n accordingly (lines **5–10**). Finally, the child nodes of n are processed recursively. If $type(n)$ represents a conditional branch or loop, the algorithm separately handles the true branch (line **13**) and the false branch (line **14**) using IF_TRUE and IF_FALSE edges. Otherwise, if n follows a sequential execution, a NEXT edge is assigned (line **17**).

Additionally, self-loop edges are added to each node to preserve intrinsic features during information aggregation (Kipf and Welling, 2016). This ensures that a node retains its original characteristics while incorporating information from its neighbors.

Fig. 2 shows the source code of a smart contract file,[1] which contains two contracts: Log and MY_BANK. In Fig. 3, the corresponding heterogeneous graph is illustrated. The FILE node represents the entire

[1] Address: 0xf015C35649c82F5467c9c74B7F28Ee67665AAD68.

Table 1
Node types and their descriptions.

Node type	Description
ENTRY_POINT	Function entry point
IF	Branch condition
RETURN	Return value
END_IF	End of branch
NEW VARIABLE	Newly defined variable in function
EXPRESSION	Expression
EXECUTION_POINT	Placeholder in custom modifiers, indicating the location of function execution
INLINE ASM	Inline assembly
BEGIN_LOOP	Loop entry
IF_LOOP	Loop condition
END_LOOP	End of loop
CONTINUE	Skip the remaining part of the current loop and move to the next iteration
LOCAL VARIABLE	Function parameter
STATE VARIABLE	Contract state variable

source file, which aggregates its internal contracts. Taking MY_BANK as an example, it includes three state variables (light cyan nodes) and three functions (blue nodes). Among them, the fallback function calls the Put function. As a result, a CALL edge (green arrow) is added to reflect this invocation. All containment relationships — such as contracts including functions or variables — are represented via AGGREGATE edges (orange arrows). To showcase intra-function modeling, we highlight the Collect function (gray dashed box). Each statement in the function body is mapped to a graph node, with types such as ENTRY_POINT, NEW_VARIABLE, LOCAL_VARIABLE, IF, EXPRESSION, and END_IF. These nodes store the corresponding source code statements as textual content. Control flow is encoded via NEXT, IF_TRUE, and IF_FALSE edges (black arrows), while data dependencies are captured using BE_READED and WRITE edges (blue arrows).

3.2.2. Initial node embedding on heterogeneous graph

In this step, we first encode the node content by converting it into a token sequence and generating vector representations for subsequent model training. To achieve this, we utilize the pretrained model GraphCodeBERT (Guo et al., 2020). GraphCodeBERT captures both semantic information at the token level and structural information at the AST (Abstract Syntax Tree) level, where the AST is a tree-based representation of source code that reflects its syntactic structure and hierarchical relationships (Yang et al., 2024). This dual representation allows for more precise node embeddings, preserving both contextual meaning and syntactic relationships.

Formally, for each node v with token sequence $S_v = \{t_1, \ldots, t_m\}$, where m denotes the length of the sequence, we compute its initial

```
contract MY_BANK {
    mapping(address => Holder) public Acc;
    Log LogFile;
    uint public MinSum = 1 ether;

    function Put(uint _unlockTime) public payable { ... }

    function Collect(uint _am) public payable {
        var acc = Acc[msg.sender];
        if (acc.balance >= MinSum && acc.balance >= _am &&
            now > acc.unlockTime) {
            if (msg.sender.call.value(_am)()) {
                acc.balance -= _am;
                LogFile.AddMessage(msg.sender, _am, "Collect"
                    );
            }
        }
    }

    function() public payable { Put(0); }
}
contract Log { ... }
```

Fig. 2. The smart contract of MY_BANK.

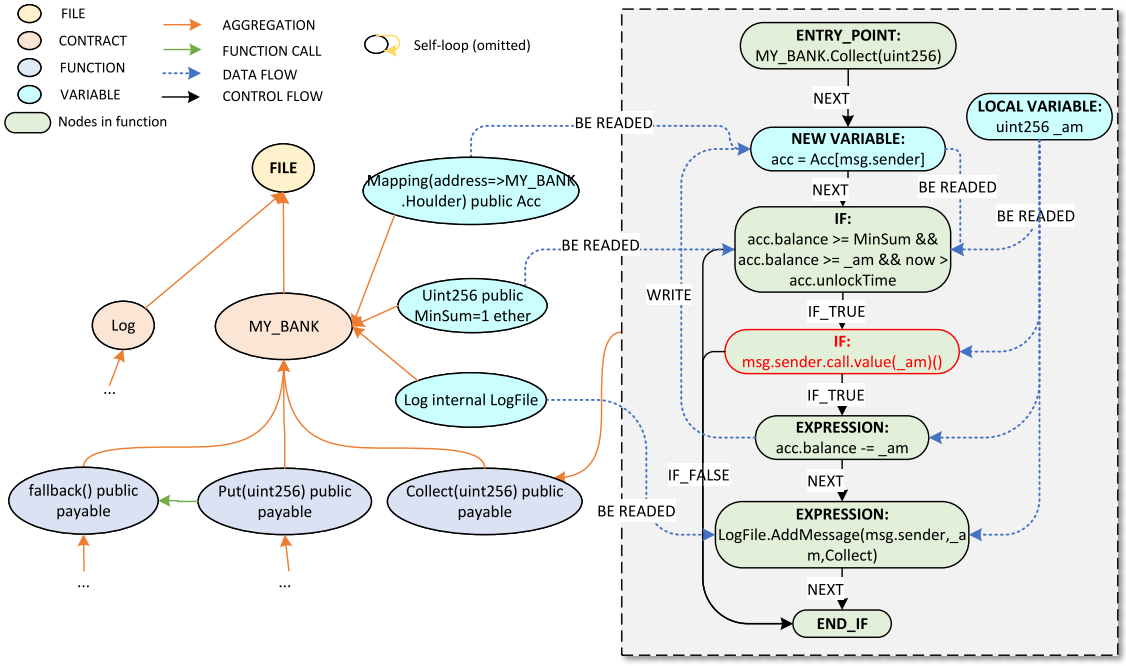


Fig. 3. The heterogeneous graph representation of the MY_BANK contract.

embedding as

$$h_v^0 = \text{GraphCodeBERT}(S_v) \in \mathbb{R}^d, \tag{1}$$

where d is the embedding dimension. This representation encodes both semantic and syntactic information and serves as the input for subsequent heterogeneous graph learning.

3.3. HGFE module

HGFE module comprises two key components: a heterogeneous graph neural network and a dual attention mechanism. The heterogeneous graph neural network learns deep representations of nodes

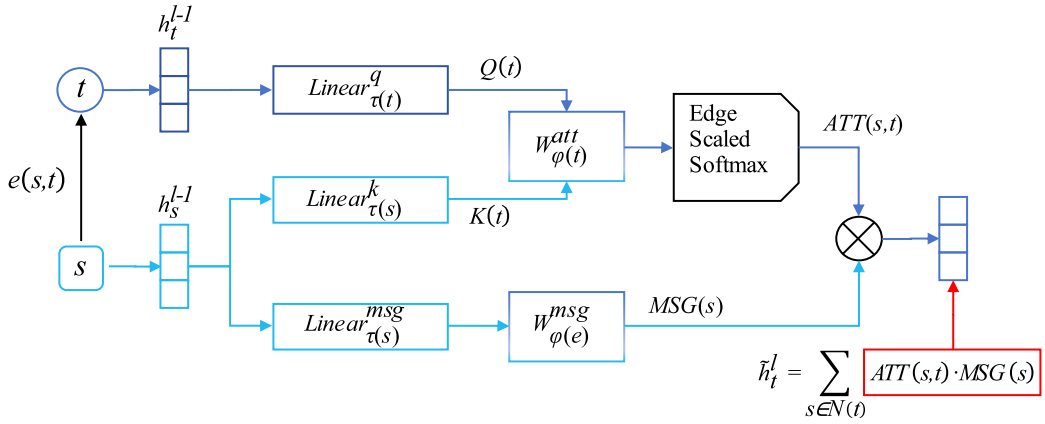


Fig. 4. Attention-based message passing mechanism in an HGT layer.

within the smart contract heterogeneous graph. It captures both structural and semantic information, enabling a more comprehensive understanding of contract behavior (Nguyen et al., 2023). To further refine the learned features, the dual attention mechanism is introduced. This mechanism highlights the importance of different nodes within the same type and assesses the contribution of various node types to vulnerability detection. In the following sections, we provide a detailed explanation of each component.

3.3.1. Heterogeneous graph neural network

We employ a multi-layer heterogeneous graph transformer (HGT) (Hu et al., 2020) as the core of our heterogeneous graph neural network. As illustrated in Fig. 4, the node representation is updated through an attention-based message passing mechanism. $\tilde{h}_t^l$, the feature representation of a node t at the lth layer, is computed as follows:

$$\tilde{h}_t^l = \sum_{s\in N(t)} \mathrm{ATT}(s,t)\cdot \mathrm{MSG}(s), \tag{2}$$

where $N(t)$ denotes the set of all neighboring nodes pointing to t. For each neighbor s, the information it transmits, $\mathrm{MSG}(s)$, is weighted by the attention score $\mathrm{ATT}(s,t)$ before being aggregated into node t. The definitions of $\mathrm{MSG}(s)$ and $\mathrm{ATT}(s,t)$ are given in Eqs. (3) and (4), respectively.

$$\mathrm{MSG}(s) = \mathrm{Linear}^{\mathrm{msg}}_{\tau(s)}(h_s^{l-1}) W^{\mathrm{msg}}_{\phi(e)}, \tag{3}$$

$$\mathrm{ATT}(s,t) = \mathrm{Softmax}\left(K(s) W^{\mathrm{att}}_{\phi(e)} Q^T(t) \cdot \frac{\mu(\tau(s),\phi(e),\tau(t))}{\sqrt{d}} \right), \tag{4}$$

where $K(s)$ and $Q(t)$ are computed as follows:

$$K(s) = \mathrm{Linear}^{k}_{\tau(s)}(h_s^{l-1}), \tag{5}$$

$$Q(t) = \mathrm{Linear}^{q}_{\tau(t)}(h_t^{l-1}). \tag{6}$$

In Eq. (3), $\tau(\cdot)$ maps a node to its corresponding node type, e denote an edge connecting s and t, $\phi(\cdot)$ maps an edge to its corresponding edge type, and h_s^{l-1} is the feature representation of node s at the $(l-1)$-th layer. The message generation process consists of two transformations: first, h_s^{l-1} is projected into a shared latent space using a node-type-specific linear transformation $\mathrm{Linear}^{\mathrm{msg}}_{\tau(s)}$; second, the resulting vector is further modulated by the edge-type-specific transformation matrix $W^{\mathrm{msg}}_{\phi(e)}$, producing the final message representation.

Similarly, in Eq. (4), the attention weight $\mathrm{ATT}(s,t)$ is derived from the compatibility between the query vector $Q(t)$ of the target node and the key vector $K(s)$ of the source node, modulated by an edge-type-aware transformation $W^{\mathrm{att}}_{\phi(e)}$. Moreover, a learnable coefficient μ is introduced to adaptively scale the attention based on the semantic importance of different meta-relation $(\tau(s),\phi(e),\tau(t))$. The normalization term $\sqrt{d}$, where d denotes the dimension of the feature vectors, ensures numerical stability in the attention computation. Finally, a softmax operation is applied across all neighbors of t to derive the normalized attention distribution.

The terms $K(s)$ and $Q(t)$ in Eq. (4) are further defined in Eqs. (5) and (6), respectively. Specifically, $K(s)$ is obtained by applying a key linear transformation $\mathrm{Linear}^{k}_{\tau(s)}$ to the feature representation h_s^{l-1} of the source node s at layer $l-1$, based on its node type $\tau(s)$. Similarly, $Q(t)$ is computed by applying a query linear transformation $\mathrm{Linear}^{q}_{\tau(t)}$ to the feature representation h_t^{l-1} of the target node t, according to its node type $\tau(t)$.

$$h_t^l = \tilde{h}_t^l + h_t^{l-1}. \tag{7}$$

To accelerate model convergence, we introduce residual connections, as shown in Eq. (7). Specifically, the final representation of node t at layer l, denoted as h_t^l, is computed by adding the extracted feature $\tilde{h}_t^l$ to the previous representation h_t^{l-1}.

3.3.2. Dual attention mechanism

After extracting deep representations for each node through L layers of HGT, we further introduce a dual attention mechanism to obtain a global representation of the heterogeneous graph. This mechanism consists of two components: intra-type attention and inter-type attention. Intra-type attention evaluates the importance of nodes within the same type for the vulnerability detection task, such as distinguishing critical functions from ordinary ones among function-type nodes. Inter-type attention models the contribution of different node types to vulnerability detection, as functions, variables, state variables, and contract nodes play distinct roles in different types of vulnerabilities.

$$h_{tp} = \sum_{i\in N_{tp}} \mathrm{Softmax}((w_2^{intra})^{\top} \cdot \mathrm{Tanh}(W_1^{intra} \cdot h_i^L)) \cdot h_i^L. \tag{8}$$

Intra-type attention computes the type-level aggregated feature h_{tp} for nodes belonging to a specific type tp. The detailed computation is shown in Eq. (8). Let N_{tp} represent the set of nodes of type tp in the heterogeneous graph. For each node i in this set, the node's feature vector h_i^L, extracted after L layers of HGT, is first projected into a new feature space through a linear transformation matrix W_1^{intra} and passed through a Tanh activation function to introduce non-linearity. Subsequently, the transformed feature is further projected by the transpose of a learnable weight vector $(w_2^{intra})^{\top}$, producing a scalar score. The attention weight for node i is then computed by applying the Softmax function for normalization. Finally, the aggregated feature h_{tp} for all nodes of type tp is obtained by performing a weighted sum over the attention-weighted features of the nodes.

$$h_{graph} = \sum_{tp\in\mathcal{A}} \mathrm{Softmax}((w_2^{inter})^{\top} \cdot \mathrm{Tanh}(W_1^{inter} \cdot h_{tp})) \cdot h_{tp}. \tag{9}$$

The inter-type attention calculation, which is used to compute the graph-level representation h_{graph}, is described in Eq. (9). The overall process is similar to that of intra-type attention, but the objects and weight matrices applied are different. Specifically, $\mathcal{A}$ represents the set of all node types in the heterogeneous graph. For each type tp in this set, we have the feature h_{tp} obtained by aggregating the intra-type attention features as described in Eq. (8). Then, a similar operation is applied to h_{tp}, where it undergoes a linear transformation using matrix W_1^{inter}, followed by a Tanh activation function and the transpose of a learnable vector $(w_2^{inter})^\top$ to compute an attention score. The attention score is then normalized using the Softmax function to calculate the attention weight. Finally, the features of all types in $\mathcal{A}$ are weighted and summed to obtain the global representation of the heterogeneous graph, h_{graph}.

3.4. VD module

VD Module is designed to perform vulnerability detection at two levels: coarse-grained contract-level detection and fine-grained line-level detection. For contract-level vulnerability detection, we pass the contract-level representation h_{graph}, extracted through the intra-type and inter-type dual attention mechanisms, into a fully connected layer. A Sigmoid activation function (Williams and Zipser, 1989) is then applied to constrain the prediction results within the range of (0,1). A threshold of 0.5 is set, where contracts with predicted values greater than 0.5 are classified as vulnerable, while those with values below 0.5 are classified as non-vulnerable. When a contract is determined to be vulnerable, we then pass the node features extracted by HGT into a fully connected layer for the line-level vulnerability detection task. Again, a Sigmoid activation function is applied to constrain the prediction results within the range of (0,1). A threshold of 0.5 is used, where nodes with a prediction greater than 0.5 are classified as vulnerable, while those with a value below 0.5 are classified as non-vulnerable. Since each node in the contract graph is associated with a specific code line, line-level vulnerability detection can be naturally achieved by directly mapping the predicted labels of nodes to the code lines they represent.

Finally, we define the overall training objective by jointly considering node-level and graph-level supervision. For node-level prediction, each node $t \in \mathcal{V}$ is associated with a ground-truth label $y_t \in \{0,1\}$ and a predicted probability $\hat{y}_t$. The node-level loss $\mathcal{L}_{\text{node}}$ is computed as the average binary cross-entropy over all nodes, encouraging the model to learn fine-grained vulnerability patterns:

$$\mathcal{L}_{\text{node}} = -\frac{1}{|\mathcal{V}|}\sum_{t\in\mathcal{V}}\left[y_t \log \hat{y}_t + (1-y_t)\log(1-\hat{y}_t)\right]. \tag{10}$$

Similarly, for graph-level prediction, each graph $G \in \mathcal{G}$ is assigned a label $y_G \in \{0,1\}$ with corresponding prediction $\hat{y}_G$. The graph-level loss $\mathcal{L}_{\text{graph}}$ is defined in the same manner to capture global structural information:

$$\mathcal{L}_{\text{graph}} = -\frac{1}{|\mathcal{G}|}\sum_{G\in\mathcal{G}}\left[y_G \log \hat{y}_G + (1-y_G)\log(1-\hat{y}_G)\right]. \tag{11}$$

To balance the contributions of these two objectives, we introduce learnable coefficients λ_{node} and λ_{graph}, which adaptively weight the node-level and graph-level losses. The overall loss $\mathcal{L}_{\text{total}}$ is then defined as:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{node}}\mathcal{L}_{\text{node}} + \lambda_{\text{graph}}\mathcal{L}_{\text{graph}}. \tag{12}$$

4. Experiment

To comprehensively evaluate the effectiveness of our proposed framework, RHDA, we design and conduct a series of systematic and in-depth experiments. These experiments aim to address the following three key research questions:

- **RQ1:** How does RHDA perform in contract-level detection?
- **RQ2:** How does RHDA perform in line-level detection?
- **RQ3:** How do different components of RHDA affect its performance?

4.1. Dataset

We use the dataset provided in Nguyen et al. (2023), which is constructed by integrating three existing datasets: SmartBugs-Curated (Durieux et al., 2020; Ferreira et al., 2020), SolidiFI-Benchmark (Ghaleb and Pattabiraman, 2020), and SmartBugs-Wild (Durieux et al., 2020; Ferreira et al., 2020). SmartBugs-Curated contains 143 smart contracts covering nine types of vulnerabilities, with some contracts containing multiple vulnerabilities. Seven common and critical types of vulnerabilities — AC, AR, DoS, FR, RE, TM, and ULLC — are selected from this dataset. SolidiFI-Benchmark includes 350 contracts with a total of 9369 injected vulnerabilities. These contracts also cover the same seven types of vulnerabilities mentioned above. SmartBugs-Wild provides 47,398 real-world Ethereum smart contracts. Using 11 different static analysis tools, 2742 vulnerability-free contracts were identified from this dataset. In total, 423 vulnerable contracts and 2742 non-vulnerable contracts are collected to construct the initial dataset. Each contract is associated with a binary label indicating whether it contains a specific type of vulnerability, supporting both contract-level and line-level vulnerability detection tasks.

Since the dataset is inherently imbalanced, directly using all samples may bias the model toward the majority class. To mitigate this issue and ensure a fair evaluation, we adopt a random under-sampling strategy, where an equal number of non-vulnerable contracts are randomly selected to match the number of vulnerable contracts in each experimental run. This strategy helps the model focus on discriminative features rather than being dominated by class distribution, which is a commonly adopted approach for handling imbalanced classification problems (Sendner et al., 2023). We acknowledge that random sampling may introduce potential bias due to the loss of information from discarded non-vulnerable samples. To alleviate this effect, the sampling process is repeated across different folds, ensuring that diverse subsets of non-vulnerable contracts are involved in training and evaluation.

Finally, we adopt 5-fold cross-validation to evaluate the model performance. The dataset is divided into five subsets. In each fold, one subset is used for testing while the remaining four are used for training. This strategy helps reduce the impact of randomness in data splitting and improves the robustness of the evaluation (Naeem and Alalfi, 2024).

4.2. Baselines

To comprehensively evaluate the performance of RHDA, we compare it with twelve representative state-of-the-art baselines. These methods are categorized based on the granularity of vulnerability detection they support: contract-level and line-level. For contract-level detection, we include the following baselines: Slither and SmartCheck (Tikhomirov et al., 2018), two traditional analysis tools; SaferSC (Tann et al., 2018), a sequence modeling-based vulnerability detection approach; ReGVD (Nguyen et al., 2022), a general graph-based vulnerability detection framework; EA-RGCN (Chen et al., 2023), which employs an edge-aware attention mechanism in relational GNNs; MRN-GCN (Liu et al., 2023), a homogeneous GNN integrating multiple relational graphs; MANDO-HGT (Nguyen et al., 2023) and MVD-HG (Xu et al., 2024), both of which utilize heterogeneous graph structures and adopt HGT (Hu et al., 2020) and RGCN (Schlichtkrull et al., 2018) as feature extractors, respectively. For line-level detection, due to the limited availability of public methods, we also include MANDO-HGT (Nguyen et al., 2023) and MVD-HG (Xu et al., 2024), which support both contract-level and line-level detection tasks. In addition, to enrich the comparison, we adopt four widely used and effective homogeneous GNNs as baselines: GCN (Kipf and Welling, 2016), GAT (Veličković et al., 2017), GraphSAGE (Hamilton et al., 2017), and GGNN (Li et al., 2015). For a fair comparison, we transform our heterogeneous contract graphs into equivalent homogeneous versions suitable for these homogeneous GNNs, and evaluate all baselines on identical dataset splits with the training hyperparameters specified in their original papers.

Table 2
Summary of experimental settings.

Category	Configuration
Embedding Dimension	768
HGT Layers	3
Dropout Rate	0.5
Learning Rate	0.0001
Optimizer	Adam
Batch Size	1/5 of dataset per vulnerability type
Epochs	50
Loss Weights	$\lambda_{\text{node}} = 0.5$, $\lambda_{\text{graph}} = 0.5$
Weight Decay	0.05
Graph Input	PyG Data (no padding)
Random Seed	Fixed (NumPy, PyTorch, CUDA)
Software	Python 3.7, PyTorch 1.31.1, CUDA 11.7
Hardware	2×RTX 4070 (16 GB), Ubuntu 22.04

4.3. Evaluation metric

We adopt the F1-score as the primary evaluation metric. This metric is particularly suitable for vulnerability detection tasks, which often suffer from class imbalance (Nguyen et al., 2023). The F1-score considers both Precision and Recall, thus providing a balanced assessment of model performance. Given True Positives (TP), False Positives (FP), and False Negatives (FN), the formulas for Precision and Recall are as follows:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{13}$$

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{14}$$

Based on these, the F1-score is computed as:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{15}$$

4.4. Experimental setup

For the experimental hyperparameter configuration, we set the dimensionality of the initial node embeddings to 768, consistent with GraphCodeBERT (Guo et al., 2020). The hidden layer size of the HGT (Hu et al., 2020) was also set to 768, with 3 layers. The detailed hyperparameter settings are summarized in Table 2. During training, the learning rate was set to 0.0001, and the Adam optimizer (Kingma and Ba, 2014) was used. The batch size refers to the number of graph samples fed into the model at each training step, which was set to one-fifth of the total data size for each vulnerability type. Both contract-level and line-level loss weights were set to 0.5. To prevent overfitting, a dropout rate of 50% was applied, and L_2 regularization was introduced with a weight decay factor of 0.05. Weight initialization follows the default PyTorch initialization for linear and embedding layers. Random seeds for NumPy, PyTorch, and CUDA were fixed to ensure reproducibility. The training was conducted over 50 epochs, and all experiments were repeated five times, with the results averaged. All graph inputs are PyTorch Geometric Data objects; no padding is applied since each graph is processed individually.

For the baseline models, we used their provided code and conducted experiments with the parameters specified in the respective papers. In terms of hardware configuration, we utilized two NVIDIA GeForce RTX 4070 GPUs, each with 16 GB of VRAM, and the operating system was Ubuntu 22.04.5 LTS. Experiments were conducted using Python 3.7, PyTorch 1.31.1, and CUDA 11.7.

4.5. Results

4.5.1. RQ1: How does RHDA perform in contract-level detection

Table 3 presents the contract-level (CL) F1-scores of RHDA compared with eight state-of-the-art neural network-based smart contract vulnerability detection methods.

Experimental results show that RHDA consistently achieves the best performance across all vulnerability types. Specifically, it achieves F1-scores of **96.26%** (AC), **97.70%** (AR), **98.95%** (DoS), **96.47%** (FR), **95.52%** (RE), **98.05%** (TM), and **90.05%** (ULLC), significantly outperforming existing approaches. The traditional detection tools, Slither and SmartCheck, perform poorly on the smart contract vulnerability detection task. In contrast, our proposed RHDA achieves at least a **14.75%** improvement in F1 score. This result highlights the significant advantages of neural network-based approaches in vulnerability detection scenarios. Compared to SaferSC, a sequence-based method, RHDA achieves F1-score improvements ranging from **9.82%** to **25.82%**, demonstrating the effectiveness of graph structures in modeling vulnerability contexts. Against ReGVD, a general GNN-based method, RHDA shows improvements of **0.75%** to **10.2%** across various vulnerabilities, highlighting its superior generalization ability. For methods based on homogeneous GNNs, such as EA-RGCN and MRN-GCN, RHDA outperforms them by **1.16%** to **34.02%**, indicating the advantage of heterogeneous graph modeling in capturing complex semantic structures within smart contracts.

Even when compared with the most recent heterogeneous GNN methods, including MANDO-HGT and MVD-HG, RHDA still achieves improvements of **0.03%** to **12.61%**, with an average gain of **5.44%**.

To evaluate the statistical significance of the performance improvements, we conduct paired t-tests between the proposed method and several representative baseline approaches based on the F1-scores across different folds. The results show that our method significantly outperforms most baselines, including MVD-HG (AC), ReGVD (AR), MRN-GCN (DoS), MRN-GCN (FR), ReGVD (RE), and MVD-HG (TM), with p-values less than 0.05, and most of them being below 0.001. However, for the ULLC category, the performance improvement is not statistically significant ($p = 0.862$), indicating that our method achieves comparable performance to the best-performing baseline on this task. This may be due to the limited performance gap between methods for this vulnerability type. Nevertheless, our method consistently outperforms this baseline on other vulnerability types, demonstrating its overall effectiveness.

These results further validate the effectiveness and superiority of our proposed heterogeneous graph construction approach and dual attention mechanism in smart contract vulnerability detection tasks. In summary, RHDA demonstrates outstanding detection capability and broad adaptability in contract-level vulnerability detection.

To evaluate the efficiency of the proposed method, we compare the end-to-end runtime with several representative and competitive baseline models, including ReGVD, MRN-GCN, MANDO-HGT, and MVD-HG. These methods are selected because they achieve strong performance and represent different types of modeling paradigms, including token-based, homogeneous graph-based, and heterogeneous graph-based approaches.

As shown in Table 4, ReGVD achieves the lowest runtime due to its lightweight representation without complex structural analysis. In contrast, graph-based methods incur additional overhead for graph construction. In particular, heterogeneous graph-based methods, such as MANDO-HGT and MVD-HG, require extracting multiple types of semantic relationships, resulting in increased runtime.

The proposed RHDA exhibits higher runtime compared to these baselines, mainly due to the construction of richer heterogeneous graphs that incorporate multiple relationships. Nevertheless, the overall processing time remains within a practical range, demonstrating a reasonable trade-off between computational efficiency and detection performance.

4.5.2. RQ2: How does RHDA perform in line-level detection

Table 5 presents the F1-score comparison of RHDA with six state-of-the-art line-level vulnerability detection methods across seven common vulnerability types.

Table 3
Comparison of RHDA and baselines on contract-level vulnerability detection.

Methods-CL	AC	AR	DoS	FR	RE	TM	ULLC
Slither (Feist et al., 2019)	78.87	76.13	77.88	78.90	75.28	78.40	75.30
SmartCheck (Tikhomirov et al., 2018)	67.06	65.92	67.15	68.22	67.62	67.11	67.14
SaferSC (Tann et al., 2018)	76.60	74.98	83.30	86.65	76.58	72.23	70.49
ReGVD (Nguyen et al., 2022)	86.06	90.73	91.97	87.68	94.77	88.47	86.41
EA-RGCN (Chen et al., 2023)	75.29	81.45	64.93	94.46	85.87	71.90	74.28
MRN-GCN (Liu et al., 2023)	86.96	88.89	95.24	95.24	80.00	91.67	88.89
MANDO-HGT (Nguyen et al., 2023)	83.65	89.41	89.89	95.23	94.78	95.99	81.75
MVD-HG (Xu et al., 2024)	91.38	89.39	89.36	93.33	89.36	96.30	90.02
Ours	**96.26**	**97.70**	**98.95**	**96.47**	**95.52**	**98.05**	**90.05**

Table 4
Runtime comparison of different methods.

Method	Runtime (ms/file)
ReGVD (Nguyen et al., 2022)	10–30
MRN-GCN (Liu et al., 2023)	50–120
MANDO-HGT (Nguyen et al., 2023)	120–200
MVD-HG (Xu et al., 2024)	150–230
Ours	220–320

The results show that RHDA achieves either the best or the second-best performance on all vulnerability types. Specifically, the F1-scores for AC, AR, DoS, FR, RE, TM, and ULLC are **98.53%**, **98.63%**, **91.46%**, **99.44%**, **97.59%**, **98.01%**, and **96.99%**, respectively. Except for DoS, RHDA outperforms all existing methods across the remaining six vulnerability types. Compared to heterogeneous GNN-based approaches such as EA-RGCN and MRN-GCN, RHDA achieves F1-score improvements ranging from **2.90%** to **15.84%**, with an average gain of **6.74%**. These results confirm the effectiveness and robustness of our proposed heterogeneous graph construction and dual attention mechanism in line-level vulnerability detection.

Due to the limited number of publicly available methods that provide fine-grained, line-level detection, we also include four commonly used homogeneous GNN baselines — GCN, GAT, GraphSAGE, and GGNN — for comparison. The experimental results indicate that, except for DoS, RHDA consistently outperforms these baselines across all other vulnerability types, with F1-score improvements ranging from **0.25%** to **23.39%**. Among the homogeneous methods, GGNN performs the best in most cases, particularly on DoS, where it surpasses RHDA by **2.58%**. However, GGNN performs worse than RHDA on most other types, with a notable **3.28%** drop in F1-score for AR.

To further evaluate the statistical significance of the performance, we conduct paired t-tests between the proposed method and the best-performing baseline (GGNN) across different vulnerability types. The results show that our method achieves statistically significant improvements on most categories, including AC, AR, FR, RE, TM, and ULLC ($p < 0.05$). However, for the DoS category, our method does not outperform GGNN, as reflected by the negative t-statistic($t = -5.83$). We speculate that this is because DoS often involve repetitive control flows or resource consumption paths. GGNN, with its strong sequence modeling capability, captures long-range dependencies and local execution patterns in the code more effectively, making it well-suited for this kind of vulnerability. In contrast, RHDA focuses on heterogeneous semantic modeling across multiple node types. It excels at integrating complex structural relationships and multi-source information, leading to better performance on vulnerabilities with intricate structures or multi-dimensional interactions.

In summary, RHDA demonstrates strong performance and broad adaptability in line-level vulnerability detection. It effectively identifies vulnerable lines within smart contracts, showcasing its practical utility in real-world scenarios.

4.5.3. RQ3: How do different modules of RHDA affect its performance

To address RQ3, we conducted systematic ablation studies to evaluate the impact of the two core innovation components in RHDA: the construction of heterogeneous smart contract graphs and the dual attention mechanism. The results of the corresponding ablation experiments are shown in Table 6 and Fig. 5.

For the heterogeneous graph construction, we designed four variants: control-flow only (CF), control-flow with data dependencies (CF+DF), control-flow with function calls (CF+FC), and control-flow with contract inheritance (CF+CI). These variants were used to explore the individual contribution of each relationship type to the overall performance. Experimental results show that RHDA consistently outperforms all variants across all vulnerability types, in both contract-level (CL) and line-level (LL) detection tasks. This further validates the effectiveness and comprehensiveness of our heterogeneous graph modeling.

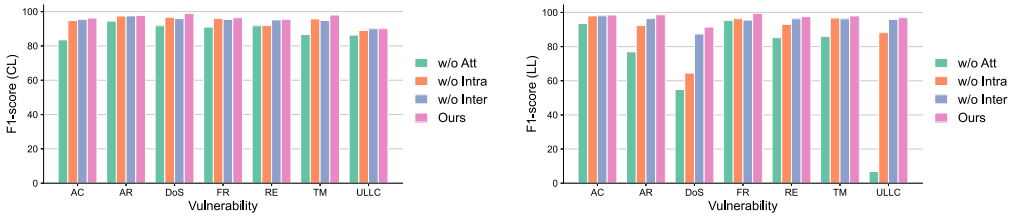
More specifically, the CF variant yielded the worst performance in most cases. Notably, it showed a performance gap of **9.47%** and **6.75%** in CL detection for DoS and RE respectively, compared to RHDA. This highlights that control-flow alone is insufficient to capture comprehensive vulnerability features. In contrast, CF+DF significantly outperformed CF in multiple types. For instance, it improved the CL and LL F1-scores for DoS by **4.85%** and **3.84%**, respectively. It also improved RE by **4.27%** at the CL level, indicating that data dependencies play a critical role in modeling variable relations and boosting detection accuracy. The CF+FC variant also showed notable gains over CF in most cases. For example, it achieved a **1.3%** higher F1-score in FR (CL), and only **0.33%** lower in FR (LL), proving the value of function call relationships in capturing inter-function semantic interactions. Similarly, CF+CI demonstrated competitive results. It notably improved DoS and RE, and achieved a **2.67%** increase in AC (CL). These results show that contract inheritance, as a key structural element, contributes positively to vulnerability modeling.

Table 5
Comparison of RHDA and baselines on line-level vulnerability detection.

Methods-LL	AC	AR	DoS	FR	RE	TM	ULLC
MANDO-HGT (Nguyen et al., 2023)	89.46	91.18	86.81	94.02	92.59	95.04	90.89
MVD-HG (Liu et al., 2023)	82.69	93.12	83.02	91.05	94.69	92.25	90.15
GCN (Kipf and Welling, 2016)	75.14	82.99	86.09	86.62	85.89	85.36	90.37
GAT (Veličković et al., 2017)	92.96	92.00	90.01	89.81	88.98	88.07	92.14
GraphSAGE (Hamilton et al., 2017)	90.67	95.02	91.21	97.94	94.31	95.90	92.35
GGNN (Li et al., 2015)	97.72	95.35	**94.04**	98.87	94.92	97.38	94.12
Ours	**98.53**	**98.63**	91.46	**99.44**	**97.59**	**98.01**	**96.99**

Table 6
Ablation study on heterogeneous graph construction.

Variants	AC		AR		DoS		FR		RE		TM		ULLC	
	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL
CF	93.49	97.34	94.27	95.41	89.52	87.56	95.09	99.34	88.77	96.51	95.02	96.67	86.64	96.51
CF+DF	95.31	97.71	96.66	96.41	94.37	91.40	95.29	99.35	93.04	97.31	97.89	**98.01**	87.14	96.59
CF+FC	95.31	97.72	94.98	97.59	95.42	91.21	96.39	99.01	93.31	97.54	96.84	97.96	87.58	96.61
CF+CI	96.16	98.01	95.87	97.32	92.94	88.95	95.29	99.37	91.01	97.20	97.89	97.98	86.94	96.58
Ours	**96.26**	**98.53**	**97.70**	**98.63**	**98.95**	**91.46**	**96.47**	**99.44**	**95.52**	**97.59**	**98.05**	**98.01**	**90.05**	**96.99**



(a) Contract-Level Detection (CL) (b) Line-Level Detection (LL)

Fig. 5. Ablation study on dual attention mechanism.

Table 7
Comparison of different heterogeneous GNN architectures on smart contract vulnerability detection.

Variants	AC		AR		DoS		FR		RE		TM		ULLC	
	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL	CL	LL
RGCN	94.44	94.86	94.70	97.75	98.85	87.15	95.29	98.01	94.68	92.55	96.84	89.83	88.05	96.89
HAN	94.52	97.50	96.66	96.82	98.82	84.38	96.05	98.43	95.07	96.32	96.94	94.19	85.01	96.84
Ours	**96.26**	**98.53**	**97.70**	**98.63**	**98.95**	**91.46**	**96.47**	**99.44**	**95.52**	**97.59**	**98.05**	**98.01**	**90.05**	**96.99**

In summary, RHDA integrates control-flow, data dependencies, function calls, and contract inheritance to construct a semantically rich heterogeneous graph. This comprehensive representation enables superior performance in multiple detection tasks, confirming the effectiveness and superiority of our graph construction strategy.

To further investigate RQ3, we design three ablation variants to evaluate the impact of the dual attention mechanism: (1) completely removing attention in the graph-level feature extraction stage (w/o Att), (2) removing intra-type attention (w/o Intra), and (3) removing inter-type attention (w/o Inter). Fig. 5 illustrates the F1-scores of these variants on both contract-level and line-level vulnerability detection tasks.

As shown in Fig. 5(a), RHDA consistently outperforms all three variants in terms of contract-level F1-score. Notably, the two variants that retain only one attention mechanism — w/o Intra and w/o Inter — still achieve significantly better performance than w/o Att, which removes all attention modules. This result confirms the effectiveness of the dual-attention mechanism in contract-level vulnerability detection.

More importantly, Fig. 5(b) reveals that the performance gain from the dual attention mechanism is more prominent in line-level detection. For example, in the ULLC vulnerability, w/o Intra, w/o Inter, and RHDA achieve approximately **80%**, **85%**, and **90%** improvements over w/o Att, respectively. This is because vulnerabilities such as ULLC often involve fine-grained semantic relations and control structure interactions across different node types. The proposed dual attention mechanism allows RHDA to better capture such critical semantic combinations, enhancing semantic aggregation and interaction during graph modeling. These enriched representations, in turn, are propagated through HGT layers, resulting in more discriminative features and stronger identification ability for vulnerable code lines. Overall, these findings highlight the collaborative and complementary benefits of intra- and inter-type attention mechanisms in multi-granularity smart contract vulnerability detection.

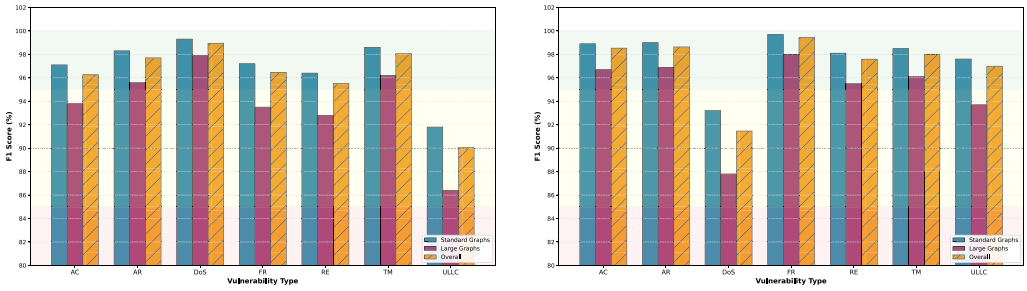
To further investigate the effectiveness of the heterogeneous graph learning architecture, we compare our model with two widely-used heterogeneous graph neural networks, namely RGCN (Schlichtkrull et al., 2018) and HAN (Wang et al., 2019), while keeping the same heterogeneous graph construction and input features. The results are presented in Table 7.

From the table, it can be observed that our method consistently outperforms both RGCN and HAN across almost all vulnerability types under both contract-level (CL) and line-level (LL) settings. Although RGCN and HAN are capable of modeling heterogeneous relations, they adopt relatively simple aggregation mechanisms, which limits their ability to capture complex semantic interactions among different node types.

In contrast, our approach leverages the HGT architecture together with the proposed dual-attention mechanism, which explicitly models both intra-type and inter-type node importance. This enables more fine-grained representation learning and leads to superior performance, particularly in more challenging scenarios such as DoS and TM at the line-level.

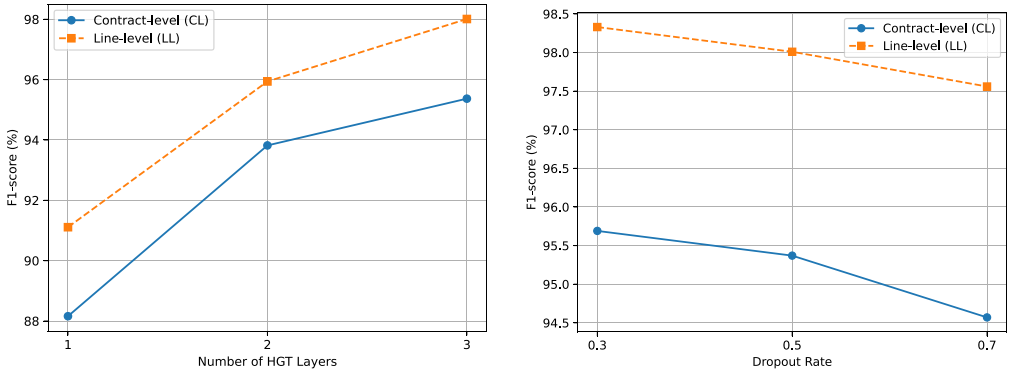
These results demonstrate that, beyond the heterogeneous graph construction itself, the choice of heterogeneous GNN architecture plays a crucial role in effectively exploiting rich semantic relationships for vulnerability detection.

To further investigate the sensitivity of the proposed model to graph size, we compare its performance on standard graphs and large graphs, as shown in Fig. 6. Specifically, based on the node count distribution across all vulnerability types, we adopt a unified threshold to distinguish graph scales. A graph is defined as a large graph if its number of nodes exceeds 375, which is determined according to the overall statistical characteristics (i.e., approximately the average of the 85th percentile and the median-plus-standard-deviation across all categories). This threshold ensures that large graphs correspond to the top 10%–15% most complex samples, while maintaining sufficient sample sizes for reliable evaluation. All remaining graphs are treated as standard graphs. Overall, the model achieves consistently better performance on standard graphs than on large graphs across all vulnerability types in both contract-level (CL) and line-level (LL) detection tasks.



(a) Contract-Level Detection (CL) (b) Line-Level Detection (LL)

Fig. 6. Performance on standard vs. large graphs.



(a) Sensitivity Analysis of HGT Layers (b) Sensitivity Analysis of Dropout Rate

Fig. 7. Hyperparameter sensitivity analysis of RHDA with respect to the number of HGT layers and dropout rate.

Specifically, in the CL setting (Fig. 6(a)), although the overall performance remains stable, large graphs consistently yield lower F1-scores than standard graphs, with performance drops ranging from approximately 1.4% to 5.4% across different vulnerability types. This observation suggests that larger graphs introduce increased structural complexity and long-range dependencies, which may weaken the model's ability to effectively capture critical vulnerability patterns.

In the LL setting (Fig. 6(b)), a similar trend is observed, where the performance gap between standard and large graphs generally falls within a moderate range (approximately 1.7%–5.4%). Most vulnerability types exhibit only minor degradation on large graphs; however, the DoS category shows a relatively larger drop (5.4%), indicating that complex control-flow structures in larger graphs pose greater challenges for fine-grained vulnerability localization.

Despite this performance gap, the proposed method maintains strong robustness across different graph scales, with only limited degradation on large graphs. This demonstrates its capability to generalize to diverse smart contract structures, including those with complex control and data dependencies.

4.6. Hyperparameter sensitivity analysis

Recent studies have shown that hyperparameter configurations play a critical role in the performance of machine learning and deep learning models, and metaheuristic optimization methods have been widely explored for automatic hyperparameter tuning (Akbulut, 2026). To evaluate the robustness of the proposed method, we conduct a sensitivity analysis on two key hyperparameters, including the number of HGT layers and the dropout rate. The average F1-scores across seven vulnerability types are reported for both contract-level (CL) and line-level (LL) detection tasks.

4.6.1. Impact of the number of HGT layers

Fig. 7(a) illustrates the performance variation with respect to different numbers of HGT layers. As the number of layers increases from 1 to 3, the performance consistently improves for both CL and LL tasks. In particular, a significant performance gain is observed when increasing the number of layers from 1 to 2, indicating that deeper models are able to capture more complex structural and semantic information in the heterogeneous graph. Further improvement is achieved with 3 layers, where the model reaches the best overall performance.

We also attempted to explore deeper architectures. However, when the number of layers exceeds 3, the computational cost increases substantially, leading to significantly slower training and higher memory consumption. In our experimental environment (16 GB GPU memory), deeper configurations often result in out-of-memory (OOM) issues, making stable training infeasible. Considering that the performance gain from increasing depth has already saturated at 3 layers, we adopt 3 layers as a practical and effective configuration that balances performance and computational efficiency.

Overall, these observations suggest that an appropriate increase in model depth enhances representation capability, while excessively deep architectures may introduce unnecessary computational overhead without clear performance benefits.

4.6.2. Impact of the dropout rate

Fig. 7(b) presents the sensitivity of the model to different dropout rates. We vary the dropout rate from 0.3 to 0.7 and observe that the

Table 8
Comparison of RHDA and different methods. ✗ indicates absence.

Methods	Techniques	Detection granularity	Number of structures
Slither (Feist et al., 2019)	IR; Taint analysis	Line-leval	✗
Smartcheck (Tikhomirov et al., 2018)	XML; XPath	Line-leval	✗
ESCORT (Sendner et al., 2023)	Transfer learning; Multi-branch classifiers	Contract-level	✗
SaferSC (Tann et al., 2018)	LSTM-based model	Contract-level	✗
ReGVD (Nguyen et al., 2022)	Token sequences; Residual connections; Language-agnostic	Function-level	✗
DA-GNN (Zhen et al., 2024)	Dual-attention mechanism(different from ours)	Contract-level	1(CFG)
EA-RGCN (Chen et al., 2023)	Function semantic graph; Residual connection; Edge attention mechanism	Function-level	3(AST+DF+CF)
MRN-GCN (Liu et al., 2023)	Multi-Relational Nested Graph Convolutional Network	Function-level	4(AST+DF+CF+Fallback)
MANDO-HGT (Nguyen et al., 2023)	Heterogeneous graphs; Graph transformer	Line-leval	2(CF+FC)
MVD-HG (Xu et al., 2024)	Heterogeneous graphs; Data augmentation	Line-leval	3(AST+CF+DF)
Our scheme	**Rich Semantic Heterograph; Dual Attention Mechanism**	**Line-leval**	**4(DF+CF+FC+CI)**

performance remains relatively stable across all settings for both CL and LL tasks. Although the best performance is achieved at a dropout rate of 0.3, the improvement over 0.5 is marginal (less than 0.5%), indicating that the model is not highly sensitive to this hyperparameter. This behavior can be attributed to the strong representation capability of the proposed framework, where the combination of heterogeneous graph modeling and dual-attention mechanisms already provides effective feature learning and implicit regularization.

In addition, a relatively higher dropout rate (e.g., 0.7) leads to a slight performance decrease, which may be caused by excessive information loss during training, especially for complex graph structures where sufficient feature propagation is required. Considering that 0.5 is a commonly adopted setting in prior studies and provides a good balance between regularization strength and model stability, we use it as the default configuration.

Overall, these results demonstrate that RHDA is robust to hyperparameter variations and can achieve strong performance without requiring extensive hyperparameter tuning.

4.7. Computational complexity analysis

In this section, we analyze the computational complexity of the proposed RHDA framework. Let N and E denote the numbers of nodes and edges in the heterogeneous graph, respectively, and d the embedding dimension. The numbers of node and edge types are fixed and relatively small, and thus can be treated as constants.

The main computational cost of RHDA arises from the heterogeneous graph learning module based on HGT. In each layer, the model performs node-wise linear transformations (e.g., query, key, and message projections) as well as edge-wise message passing with attention aggregation. The linear transformations introduce a computational cost proportional to d^2, while the message passing and attention operations scale with the number of edges. Therefore, the complexity of a single HGT layer can be approximated as $O((N+E)\cdot d^2)$. Considering L stacked layers, the total complexity becomes $O(L \cdot (N+E) \cdot d^2)$.

The proposed dual-attention mechanism introduces additional computations for graph-level representation learning. Specifically, the intra-type attention operates over all nodes, while the inter-type attention is applied over a small number of node types. Since the number of node types is fixed, the overall cost of this module is relatively small compared to the graph learning component.

In summary, the computational complexity of RHDA is dominated by the HGT-based graph propagation and scales approximately linearly with respect to the size of the graph. This indicates that the proposed method is efficient and applicable to large-scale smart contract analysis.

5. Related work

To provide a clear comparison of different vulnerability detection methods, we summarize their key characteristics in Table 8. In the following, we will provide a detailed overview of the related work in the field of smart contract vulnerability detection.

5.1. Traditional methods for smart contract vulnerability detection

Traditional methods for smart contract vulnerability detection primarily rely on techniques such as static analysis, dynamic analysis, symbolic execution, fuzz testing, formal verification, and taint analysis to identify potential security flaws. For instance, Slither (Feist et al., 2019) converts Solidity smart contracts into an intermediate representation, SlithIR, and uses Static Single Assignment (SSA) form along with a simplified instruction set to perform data flow and taint tracking analysis. ContractFuzzer (Jiang et al., 2018), based on fuzz testing, automatically generates input test cases, deploying and executing contracts to discover vulnerabilities such as Denial of Service (DoS) and improper exception handling. Echidna (Grieco et al., 2020) employs symbolic execution path exploration and fuzz testing to generate inputs, simulating contract execution paths to detect vulnerabilities like overflow and state consistency errors. SmartCheck (Tikhomirov et al., 2018) converts Solidity code into an XML-based intermediate representation and uses XPath patterns to analyze relationships between elements, uncovering program vulnerabilities. However, these methods primarily rely on static rules, simplified analysis models, or limited runtime information. As a result, they struggle to fully capture the complex logic and dynamic behavior inherent in smart contracts. Their contextual understanding is limited, leading to false positives and negatives. Moreover, they lack sufficient adaptability when dealing with novel vulnerabilities.

5.2. Neural network-based vulnerability detection methods

Neural network-based vulnerability detection methods can be broadly classified into sequence-based and graph-based approaches. Sequence-based methods, such as SaferSC (Tann et al., 2018), leverage Long Short-Term Memory (LSTM) networks to model opcode sequences of smart contracts. These models capture contextual relationships within the code to identify potential vulnerabilities. ESCORT (Sendner et al., 2023) extracts semantic features from bytecode using deep neural networks, forms unified representations, and builds a multi-task learning framework to detect various types of vulnerabilities in parallel. ReGVD (Nguyen et al., 2022) treats source code as a flat token sequence, constructs graphs using token embeddings from a pre-trained language model, and applies residual GNN layers with mixed pooling to classify code snippets for vulnerability detection. Although sequence-based methods are effective in handling linear data, they often overlook the structural information of programs. As a result, they lack the capacity to model complex contexts and exhibit limited generalization capabilities. These limitations hinder their performance in comprehensive vulnerability detection tasks.

Graph-based methods can be further divided into homogeneous and heterogeneous graph approaches. Homogeneous graph-based methods, such as EA-RGCN (Chen et al., 2023), model smart contract functions as semantic graphs (SG). They extract content and semantic features of the code by introducing edge attention mechanisms and residual graph

convolutional networks (RGCN). MRN-GCN (Liu et al., 2023) leverages multi-relational graphs at both the function and contract levels. It combines graph convolution and attention mechanisms to capture structural and semantic information. As a result, it enables fine-grained vulnerability detection at the function level. DA-GNN (Zhen et al., 2024) combines control flow graph construction, feature extraction, dual attention mechanisms, and graph feature fusion to propose an efficient and accurate method for smart contract vulnerability detection. These homogeneous graph-based neural network methods model smart contracts as uniformly structured graphs. However, they overlook the heterogeneity of nodes and edges, making it difficult for the model to capture the complex semantic information and contextual dependencies within contracts.

Heterogeneous graph-based neural network methods construct graph structures that include various node and edge types, offering a more comprehensive representation of the complex semantic information in smart contracts. For example, MANDO-HGT (Nguyen et al., 2023) constructs a heterogeneous contract graph incorporating control flow and function call information. By introducing the HGT model and learning the meta-relations between different nodes and edges, it enables the detection and localization of various vulnerabilities in smart contracts. MVD-HG (Xu et al., 2024), on the other hand, fuses abstract syntax trees with control flow and data flow graphs to create a heterogeneous graph. Using graph neural networks, it achieves multi-granularity vulnerability detection at both contract-level and line-level, enhancing detection accuracy and generalization ability. However, these heterogeneous graph-based methods still have limitations. When constructing the heterogeneous graph, the semantic relationships considered are not comprehensive enough, such as the lack of representation for higher-level structural semantics like contract inheritance. Furthermore, during the feature extraction phase, they fail to fully exploit the semantic heterogeneity among different node types, which affects the model's ability to recognize complex vulnerability patterns. Therefore, constructing a heterogeneous contract graph that integrates rich structural relationships and fully exploits the graph's heterogeneity is crucial. In addition, recent studies have shown that metaheuristic optimization methods can also provide effective solutions for complex high-dimensional optimization tasks and hyperparameter tuning problems (Akbulut and Aslantas, 2025), which may further benefit graph-based vulnerability detection models.

6. Discussion

6.1. Real-world deployment considerations

To further assess the practical applicability of the proposed method, we discuss its scalability and integration with existing static analysis tools.

Scalability. The proposed approach operates on graph representations derived from smart contract bytecode, where the computational complexity mainly depends on the number of nodes and edges. Benefiting from the efficiency of graph neural networks, the model scales reasonably well to larger graphs. This is also supported by our experimental results on large graphs, where the performance degradation remains moderate, indicating stable behavior under increasing structural complexity. Moreover, the inference process can be efficiently parallelized, making the method suitable for large-scale contract analysis scenarios.

Integration with existing tools. The proposed method operates directly on smart contract bytecode, treating the internal analysis process as a black-box model from the user perspective. In practice, users only need to provide compiled bytecode as input and obtain vulnerability detection results as output, without requiring access to or modification of intermediate representations or underlying analysis procedures. This design significantly simplifies integration with existing toolchains, as the model can be easily incorporated as an independent detection module. It can be deployed as a standalone analysis service or integrated into existing security pipelines for automated contract auditing.

Overall, these characteristics demonstrate that the proposed approach is not only effective in terms of detection performance but also practical for real-world deployment in smart contract security analysis.

6.2. Limitations

While the proposed method achieves strong performance, several limitations remain, many of which are common in existing learning-based smart contract analysis approaches. First, RHDA is based on static analysis of smart contract bytecode. Similar to most existing static analysis and learning-based methods, it may not fully capture certain dynamic execution behaviors involved in complex vulnerabilities. Second, the current study focuses on seven representative types of smart contract vulnerabilities. This setting is consistent with prior works that typically evaluate on a fixed set of well-studied vulnerabilities, while the extensibility of the proposed framework to a broader range of vulnerability types remains to be further explored. These limitations suggest promising directions for future work. In particular, incorporating dynamic analysis signals and extending the framework to support a wider range of vulnerability types could further enhance the applicability and effectiveness of the proposed method in real-world scenarios.

Furthermore, recent studies have explored metaheuristic optimization methods for solving complex optimization and hyperparameter tuning problems (Akbulut, 2025; Akbulut et al., 2025). Given that the proposed framework involves multiple components, including heterogeneous graph construction and dual-attention mechanisms, incorporating such optimization strategies may provide a more systematic way to tune key hyperparameters, such as embedding dimensions and network depth. We leave the integration of metaheuristic optimization techniques as an interesting direction for future work.

7. Conclusion

Smart contracts play a critical role in decentralized applications, yet their vulnerabilities may lead to severe financial and security risks. Existing detection methods, especially graph-based approaches, still suffer from limited semantic modeling and insufficient exploitation of heterogeneous structures, which restrict their overall effectiveness. In this paper, we propose RHDA, a novel smart contract vulnerability detection framework based on rich heterogeneous graph representations and a dual-attention mechanism. By integrating multiple semantic relationships, including control flow, data dependencies, function calls, and contract inheritance, RHDA constructs a more expressive representation of smart contracts. Furthermore, the proposed dual-attention mechanism enables fine-grained modeling of both intra-type and inter-type node importance, enhancing the model's ability to capture complex vulnerability patterns.

Extensive experimental results on seven representative vulnerability types demonstrate that RHDA consistently outperforms existing methods on both contract-level and line-level detection tasks. In particular, the proposed method achieves performance gains of up to 34.02% compared to baseline methods, and attains F1-scores around or above 95% across most vulnerability categories. In addition, RHDA shows stable performance across different graph scales, indicating strong robustness and generalization capability. The computational complexity scales approximately linearly with the graph size, ensuring its practicality for large-scale smart contract analysis. Despite these promising results, there is still room for further improvement. Future work will focus on enhancing the modeling capability and extending the applicability of the proposed framework to more complex and diverse vulnerability scenarios.

CRediT authorship contribution statement

Yingying Qu: Writing – original draft, Methodology, Conceptualization. **Jiangtao Cui:** Validation, Data curation. **Haotian Chi:** Investigation, Data curation. **Yonggang Li:** Supervision, Resources. **Yeh-Ching Chung:** Resources. **Shunrong Jiang:** Writing – review & editing, Supervision, Resources.

Declaration of competing interest

The authors have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62302282), the Shanxi Province Science Foundation (202203021222005) and the Fundamental Research Funds for the Central Universities of Ministry of Education of China (2023QN1078).

Data availability

Data will be made available on request.

References

Akbulut, H., 2025. Meta-heuristic optimization for optimal block size in multi-focus image fusion: a comprehensive comparative studys. Vis. Comput. 41 (13), 11025–11051.

Akbulut, H., 2026. A modified starfish optimization algorithm (M-SFOA) for global optimization problems and its application to heart disease risk prediction. Expert Syst. Appl. 131088.

Akbulut, H., Aslantas, V., 2025. An optimal multi-exposure image fusion using genetic algorithm. Arab. J. Sci. Eng. 1–25.

Akbulut, H., Atasever, S., Sıramkaya, E., 2025. Machine learning-based orange quality classification: A hyperparameter optimization approach through puma optimizer. In: 2025 7th International Congress on Human-Computer Interaction, Optimization and Robotic Applications. ICHORA, IEEE, pp. 1–5.

Atzei, N., Bartoletti, M., Cimoli, T., 2017. A survey of attacks on ethereum smart contracts (sok). In: International Conference on Principles of Security and Trust. Springer, pp. 164–186.

Bresil, M., Prasad, P., Sayeed, M.S., Bukar, U.A., 2025. Deep learning-based vulnerability detection solutions in smart contracts: A comparative and meta-analysis of existing approaches. IEEE Access 13, 28894–28919. http://dx.doi.org/10.1109/ACCESS.2025.3532326.

Buterin, V., 2014. A next-generation smart contract and decentralized application platform. White Pap. 3 (37), 1–36.

Buterin, V., et al., 2013. Ethereum white paper. GitHub Repos. 1 (22–23), 5–7.

Chen, D., Feng, L., Fan, Y., Shang, S., Wei, Z., 2023. Smart contract vulnerability detection based on semantic graph and residual graph convolutional networks with edge attention. J. Syst. Softw. 202, 111705.

Crincoli, G., Iadarola, G., La Rocca, P.E., Martinelli, F., Mercaldo, F., Santone, A., 2022. Vulnerable smart contract detection by means of model checking. In: Proceedings of the Fourth ACM International Symposium on Blockchain and Secure Critical Infrastructure. pp. 3–10.

Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P., 2020. Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. pp. 530–541.

Fei, J., Chen, X., Zhao, X., 2023. MSmart: Smart contract vulnerability analysis and improved strategies based on smartcheck. Appl. Sci. 13 (3), 1733.

Feist, J., Grieco, G., Groce, A., 2019. Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain. WETSEB, IEEE, pp. 8–15.

Ferreira, J.F., Cruz, P., Durieux, T., Abreu, R., 2020. Smartbugs: A framework to analyze solidity smart contracts. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. pp. 1349–1352.

Gers, F.A., Schmidhuber, J., Cummins, F., 2000. Learning to forget: Continual prediction with LSTM. Neural Comput. 12 (10), 2451–2471.

Ghaleb, A., Pattabiraman, K., 2020. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 415–427.

Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A., 2020. Echidna: effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 557–560.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2020. Graphcodebert: Pre-training code representations with data flow. arXiv preprint arXiv:2009.08366.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. In: NIPS'17, pp. 1025–1035.

Han, K., Xiao, A., Wu, E., Guo, J., Xu, C., Wang, Y., 2021. Transformer in transformer. Adv. Neural Inf. Process. Syst. 34, 15908–15919.

Hu, Z., Dong, Y., Wang, K., Sun, Y., 2020. Heterogeneous graph transformer. In: Proceedings of the Web Conference 2020. pp. 2704–2710.

Jiang, B., Liu, Y., Chan, W.K., 2018. Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. pp. 259–269.

Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

Kipf, T.N., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

Li, W., Li, X., Mao, Y., Zhang, Y., 2026. Interaction-aware vulnerability detection in smart contract bytecodes. IEEE Trans. Dependable Secur. Comput. 23 (1), 298–315.

Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R., 2015. Gated graph sequence neural networks. arXiv preprint arXiv:1511.05493.

Liang, R., Chen, J., Wu, C., He, K., Wu, Y., Cao, R., Du, R., Zhao, Z., Liu, Y., 2025. Vulseye: Detect smart contract vulnerabilities via stateful directed graybox fuzzing. IEEE Trans. Inf. Forensics Secur. 20, 2157–2170. http://dx.doi.org/10.1109/TIFS.2025.3537827.

Liu, H., Fan, Y., Feng, L., Wei, Z., 2023. Vulnerable smart contract function locating based on multi-relational nested graph convolutional network. J. Syst. Softw. 204, 111775.

Naeem, H., Alalfi, M.H., 2024. Machine learning for cross-vulnerability prediction in smart contracts. In: 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops. ICSTW, IEEE, pp. 21–28.

Nguyen, V.-A., Nguyen, D.Q., Nguyen, V., Le, T., Tran, Q.H., Phung, D., 2022. Regvd: Revisiting graph neural networks for vulnerability detection. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. pp. 178–182.

Nguyen, H.H., Nguyen, N.-M., Xie, C., Ahmadi, Z., Kudendo, D., Doan, T.-N., Jiang, L., 2023. Mando-hgt: Heterogeneous graph transformers for smart contract vulnerability detection. In: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories. MSR, IEEE, pp. 334–346.

Niu, Z., Zhong, G., Yu, H., 2021. A review on the attention mechanism of deep learning. Neurocomputing 452, 48–62.

Qu, Y., Cui, J., Chi, H., Geng, H., Jiang, S., Du, X., Meng, W., 2025. Smart contract vulnerability detection via heterogeneous graph representation and dual attention mechanisms. In: 2025 IEEE Global Communications Conference, GLOBECOM 2025, Taipei, Taiwan, December 8-12, 2025. pp. 2916–2921.

Ren, S., He, L., Tu, T., Wu, D., Liu, J., Ren, K., Chen, C., 2025. Lookahead: Preventing defi attacks via unveiling adversarial contracts. Proc. ACM Softw. Eng. 2 (FSE), 1847–1869.

Samreen, N.F., Alalfi, M.H., 2021. Smartscan: an approach to detect denial of service vulnerability in ethereum smart contracts. In: 2021 IEEE/ACM 4th International Workshop on Emerging Trends in Software Engineering for Blockchain. WETSEB, IEEE, pp. 17–26.

Schlichtkrull, M., Kipf, T.N., Bloem, P., Van Den Berg, R., Titov, I., Welling, M., 2018. Modeling relational data with graph convolutional networks. In: The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings 15. Springer, pp. 593–607.

Sendner, C., Chen, H., Fereidooni, H., Petzi, L., König, J., Stang, J., Dmitrienko, A., Sadeghi, A.-R., Koushanfar, F., 2023. Smarter contracts: Detecting vulnerabilities in smart contracts with deep transfer learning. In: NDSS 2023.

Sun, Y., Huang, S., Li, G., Chen, R., Liu, Y., Jiang, Q., 2023. A smart contract vulnerability detection method based on program dependency graph. In: 2023 4th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+AI). IEEE, pp. 84–89.

Szabo, N., 1996. Smart contracts: building blocks for digital markets. EXTROPY: J. Transhumanist Thought,(16) 18 (2), 28.

Tann, W.J.-W., Han, X.J., Gupta, S.S., Ong, Y.-S., 2018. Towards safer smart contracts: A sequence learning approach to detecting security threats. arXiv preprint arXiv:1811.06632.

Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y., 2018. Smartcheck: Static analysis of ethereum smart contracts. In: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain. pp. 9–16.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.

Wang, P., Bai, L., Yang, X., Wang, X., Liang, J., 2025. Generalization error bounds for graph neural networks in unseen domains. IEEE Trans. Artif. Intell. 6 (10), 2712–2721. http://dx.doi.org/10.1109/TAI.2025.3556978.

Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., Yu, P.S., 2019. Heterogeneous graph attention network. In: The World Wide Web Conference. pp. 2022–2032.

Williams, R.J., Zipser, D., 1989. A learning algorithm for continually running fully recurrent neural networks. Neural Comput. 1 (2), 270–280.

Xu, J., Wang, T., Lv, M., Chen, T., Zhu, T., Ji, B., 2024. MVD-HG: multigranularity smart contract vulnerability detection method based on heterogeneous graphs. Cybersecurity 7 (1), 55.

Yang, Y., Xu, H., Yao, S., 2024. Enhancing smart contract vulnerability detection with hybrid pre-trained models on source code and byte code. In: 2024 6th International Conference on Frontier Technologies of Information and Computer. ICFTIC, IEEE, pp. 122–126.

Zhang, Y., Liu, D., 2022. Toward vulnerability detection for ethereum smart contracts using graph-matching network. Futur. Internet 14 (11), 326.

Zhao, Z., Liu, Z., Wang, Y., Yang, D., Che, W., 2024. RA-HGNN: Attribute completion of heterogeneous graph neural networks based on residual attention mechanism. Expert Syst. Appl. 243, 122945.

Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H., 2024. DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network. Comput. Netw. 242, 110238.